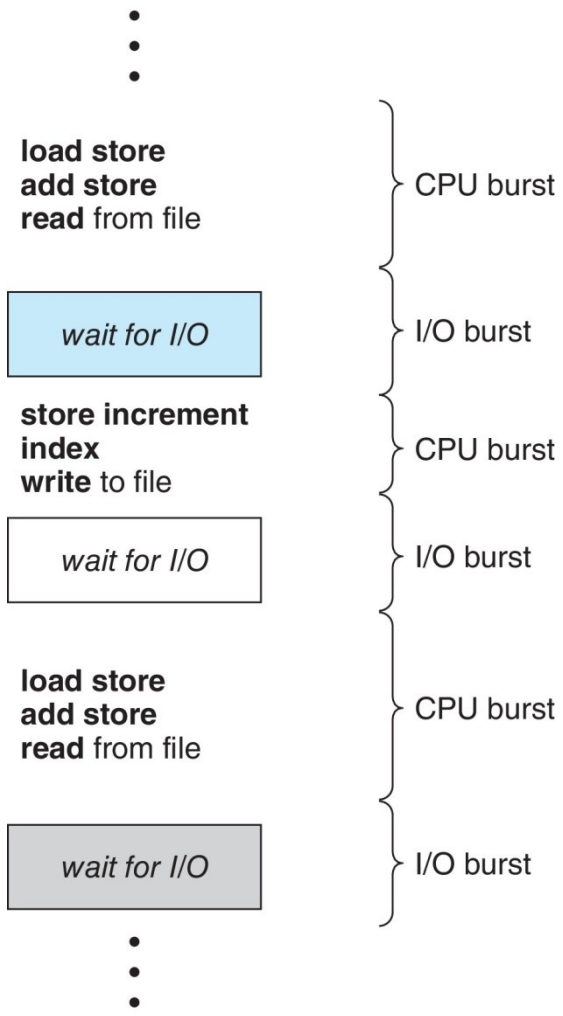
processor Scheduling

CPU Scheduling

- **We concentrate on the problem of scheduling the usage of a single processor among all the existing processes in the system**
- **The goal is to achieve**
  - High processor utilization
  - High throughput
    - number of processes completed per unit time

Basic Concepts

- Maximum CPU utilization obtained with multiprogramming
- CPU–I/O Burst Cycle – Process execution consists of a **cycle** of CPU execution and I/O wait
- **CPU burst** followed by **I/O burst**
- CPU burst distribution is of main concern



CPU Scheduler

- **The CPU scheduler selects from among the processes in ready queue, and allocates a CPU core to one of them**
  - Queue may be ordered in various ways
- **CPU scheduling decisions may take place when a process:**
  1. Switches from running to waiting state
  2. Switches from running to ready state
  3. Switches from waiting to ready
  4. Terminates

Preemptive and Nonpreemptive Scheduling

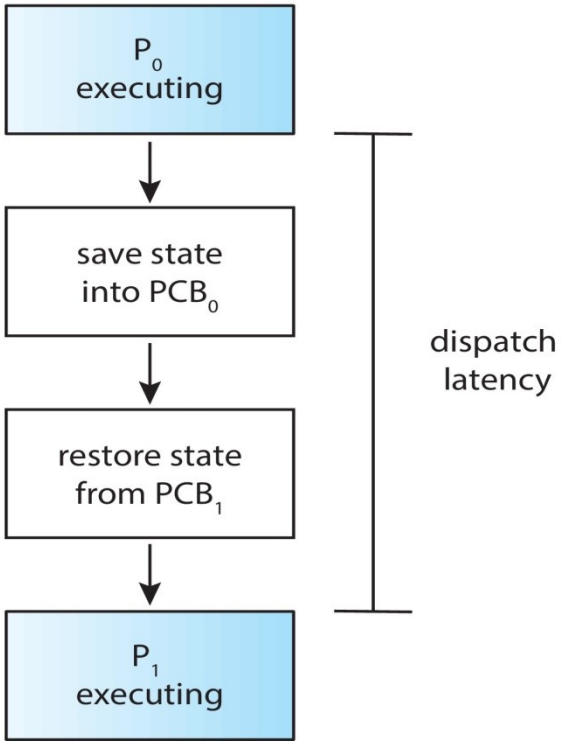
- Under Nonpreemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases it either by terminating or by switching to the waiting state.
- Virtually all modern operating systems including Windows, MacOS, Linux, and UNIX use preemptive scheduling algorithms.

Preemptive Scheduling and Race Conditions

- Preemptive scheduling can result in race conditions when data are shared among several processes.
- Consider the case of two processes that share data. While one process is updating the data, it is preempted so that the second process can run. The second process then tries to read the data, which are in an inconsistent state.

Dispatcher

- **Dispatcher module gives control of the CPU to the process selected by the CPU scheduler; this involves:**
  - Switching context
  - Switching to user mode
  - Jumping to the proper location in the user program to restart that program
- **Dispatch latency – time it takes for the dispatcher to stop one process and start another running**



Scheduling Criteria

- CPU utilization – keep the CPU as busy as possible
- Throughput – # of processes that complete their execution per time unit
- Turnaround time – amount of time to execute a particular process
- Waiting time – amount of time a process has been waiting in the ready queue
- Response time – amount of time it takes from when a request was submitted until the first response is produced.

Scheduling Algorithm Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

Priorities

- Implemented by having multiple ready queues to represent each level of priority
- Scheduler will always choose a process of higher priority over one of lower priority
- Lower-priority may suffer starvation
- Then allow a process to change its priority based on its age or execution history
- Our first scheduling algorithms will not make use of priorities
- We will then present other algorithms that use dynamic priority mechanisms

Characterization of Scheduling Policies

- **The selection function: determines which process in the ready queue is selected next for execution**
- **The decision mode: specifies the instants in time at which the selection function is exercise**
  - Nonpreemptive
    - Once a process is in the running state, it will continue until it terminates or blocks itself for I/O
  - Preemptive
    - Currently running process may be interrupted and moved to the Ready state by the OS
    - Allows for better service since any one process cannot monopolize the processor for very long

The CPU-I/O Cycle

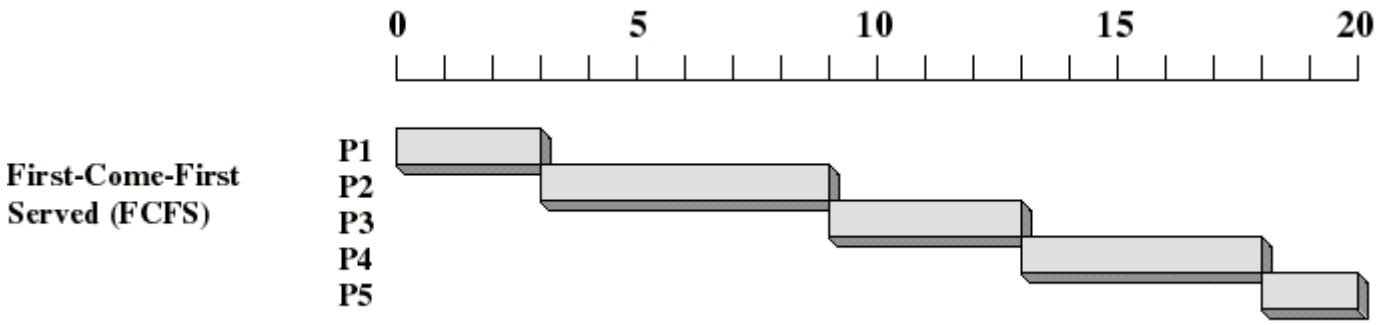
- We observe that processes require alternate use of processor and I/O in a repetitive fashion
- Each cycle consist of a CPU burst (typically of 5 ms) followed by a (usually longer) I/O burst
- A process terminates on a CPU burst
- CPU-bound processes have longer CPU bursts than I/O-bound processes

Scheduling

The part of operating system which makes a choice that which process to run next is called scheduler.

The algorithm it uses is called the scheduling algorithm.

SCHEDULING ALGORITHMS
First Come First Served (FCFS)



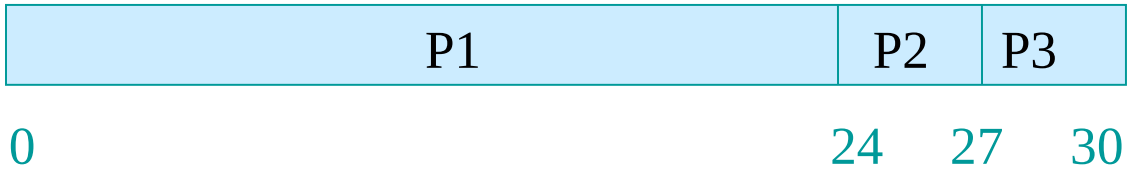
- **Selection function: the process that has been waiting the longest in the ready queue (hence, FCFS)**
- **Decision mode: nonpreemptive**
  - a process run until it blocks itself

- **Completion Time**: Time taken for the execution to complete, starting from arrival time.
- **Turn Around Time**: Time taken to complete after arrival. In simple words, it is the difference between the Completion time and the Arrival time.
- **Waiting Time**: Total time the process has to wait before it's execution begins. It is the difference between the Turn Around time and the Burst time of the process.

First Come First Served (FCFS)

Process	CPU Burst Time (ms)
P1	24
P2	3
P3	3

Suppose that processes arrive in the order: P1, P2, P3, we get the result shown in the Gantt chart below:



Waiting time=finish time-burst time-arrival time

P1-w.t=24-24-0=0 p2-w.t=27-3-0=24 p3-w.t=30-3-0=27

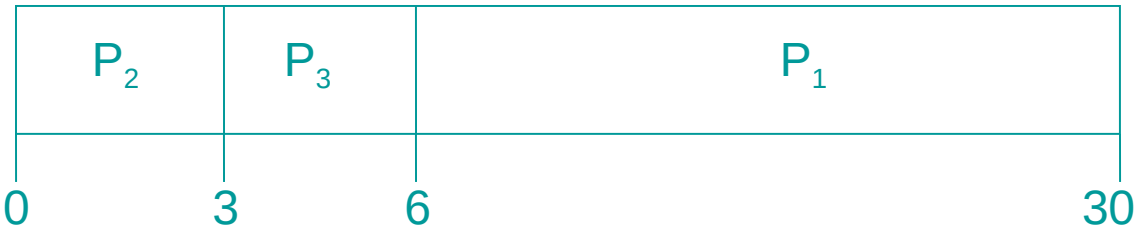
Waiting time for P1 = 0; P2 = 24; P3 = 27

FCFS Scheduling (Cont.)

Suppose that the processes arrive in the order

P_2 , P_3 , P_1 .

- The Gantt chart for the schedule is:



- W.T=F.T-B.T-Arrival Time
- Waiting time for $P_1 = 6$; $P_2 = 0$, $P_3 = 3$
- Average waiting time: (6 + 0 + 3)/3 = 3
- Much better than previous case.
- *Convoy effect* short process behind long process

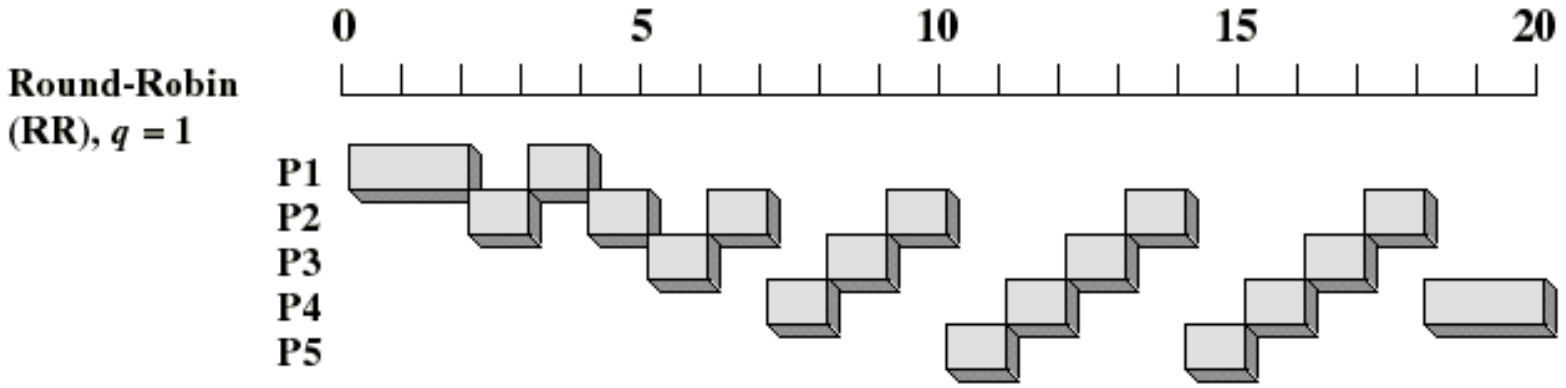
FCFS drawbacks/Convoy effect?

- **A process that does not perform any I/O will control the processor**
- **Favors CPU-bound processes**
  - I/O-bound processes have to wait until CPU-bound process completes
  - They may have to wait even when their I/O are completed (poor device utilization)

Round-Robin

- CPU is assigned to the process on the basis of FCFS for a fixed amount of time.
- This fixed amount of time is called as time quantum or time slice.
- After the time quantum expires, the running process is preempted and sent to the ready queue.
- Then, the processor is assigned to the next arrived process.
- It is always preemptive in nature.

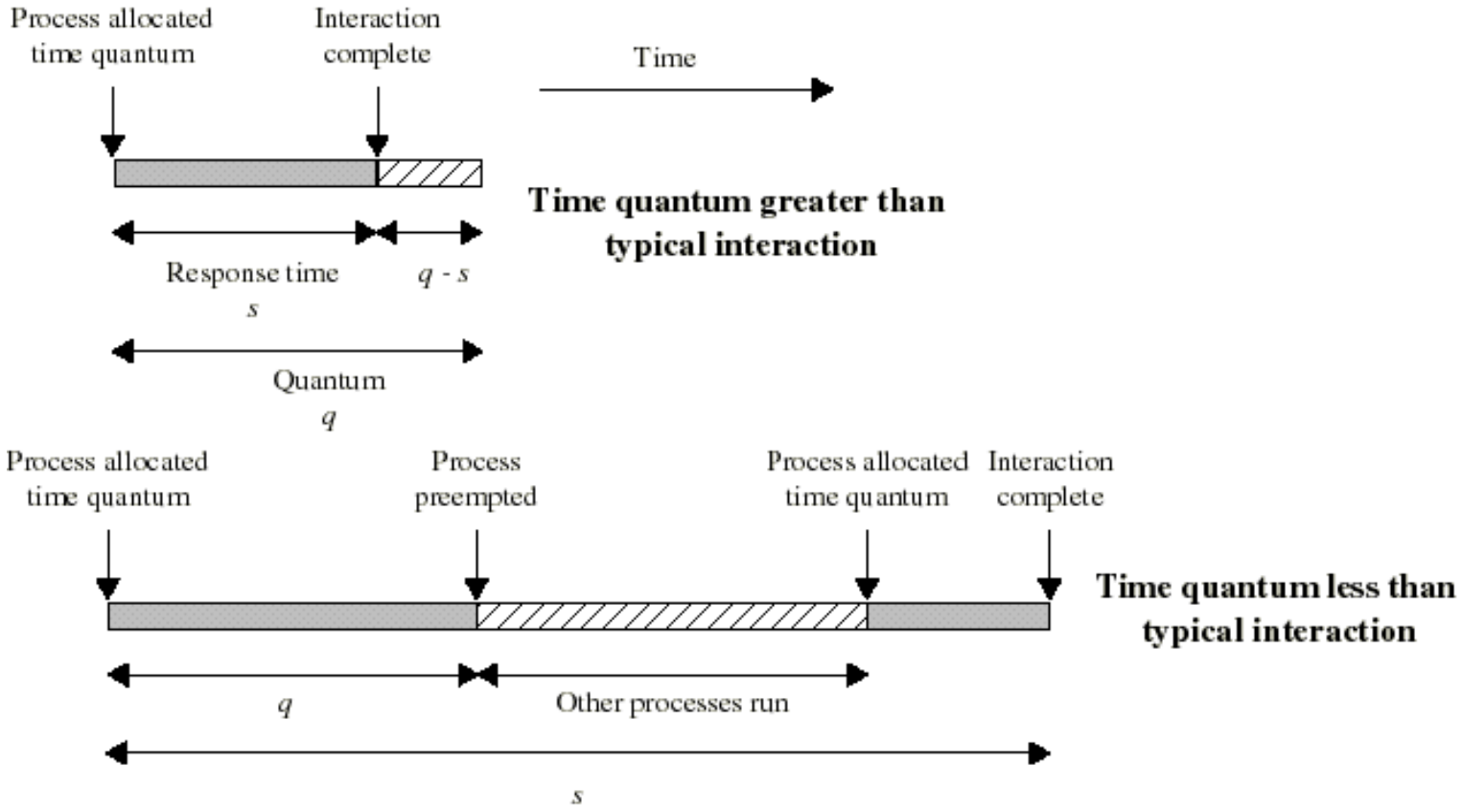
Round-Robin



- **Selection function: same as FCFS**
- **Decision mode: preemptive**
  - a process is allowed to run until the time slice period (quantum, typically from 10 to 100 ms) has expired
  - then a clock interrupt occurs and the running process is put on the ready queue

Time Quantum for Round Robin

- must be significantly larger than the time required to handle the clock interrupt and dispatching
- should be larger then the typical interaction (but not much more to avoid penalizing I/O bound processes)



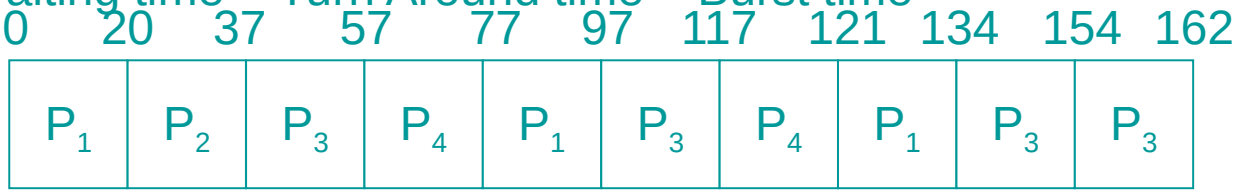
Example: RR with Time Quantum = 20

Process	Burst Time R.t
P_1	53-20=33
P_2	17
P_3	68
P_4	24

- **The Gantt chart is:**

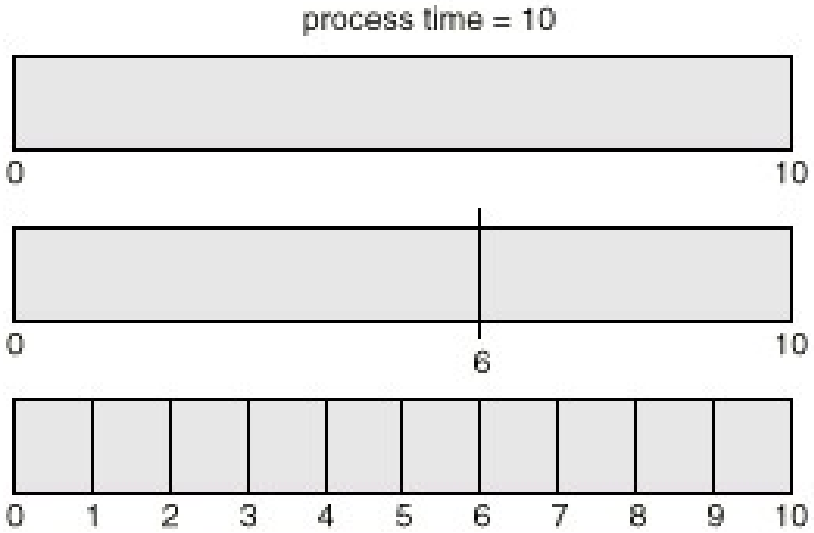
  Turn Around time = Exit time – Arrival time

- Waiting time = Turn Around time – Burst time



- **W.T=F.T-B.T-Arrival Time**
- **Typically, higher average turnaround than SJF, but better**

How a Smaller Time Quantum Increases Context Switches



quantum	context switches
12	0
6	1
1	9

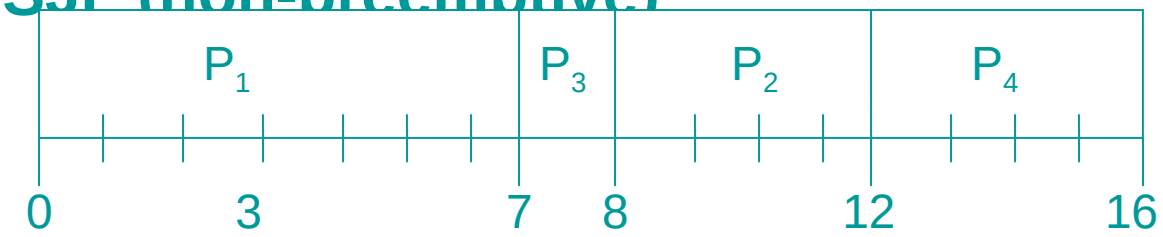
Shortest-Job-First (SJR) Scheduling

- **Associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest time.**
- **Two schemes:**
  - nonpreemptive – once CPU given to the process it cannot be preempted until completes its CPU burst.
  - Preemptive – if a new process arrives with CPU burst length less than remaining time of current executing process, preempt. This scheme is know as the Shortest-Remaining-Time-First (SRTF).
- **SJF is optimal – gives minimum average waiting time for a given set of processes.**

Example of Non-Preemptive SJF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (non-preemptive)

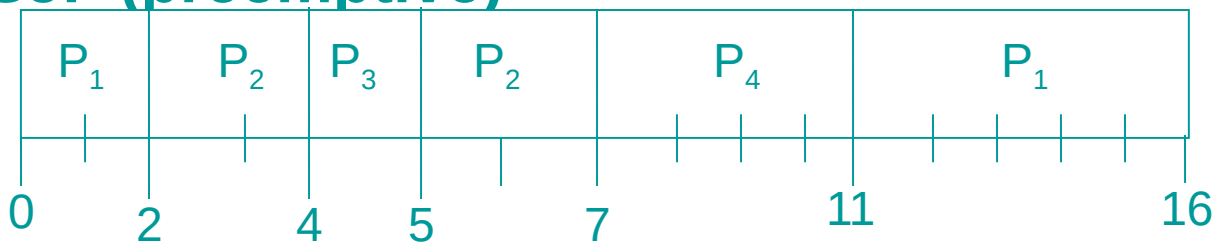


- Average waiting time = (0 + 6 + 3 + 7)/4 - 4

Example of Preemptive SJF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF (preemptive)



- Average waiting time = (9 + 1 + 0 +2)/4

Priority Scheduling

- **A priority number (integer) is associated with each process**
- **The CPU is allocated to the process with the highest priority (smallest integer ≡ highest priority).**
  - Preemptive
  - nonpreemptive

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

- Priority scheduling Gantt Chart

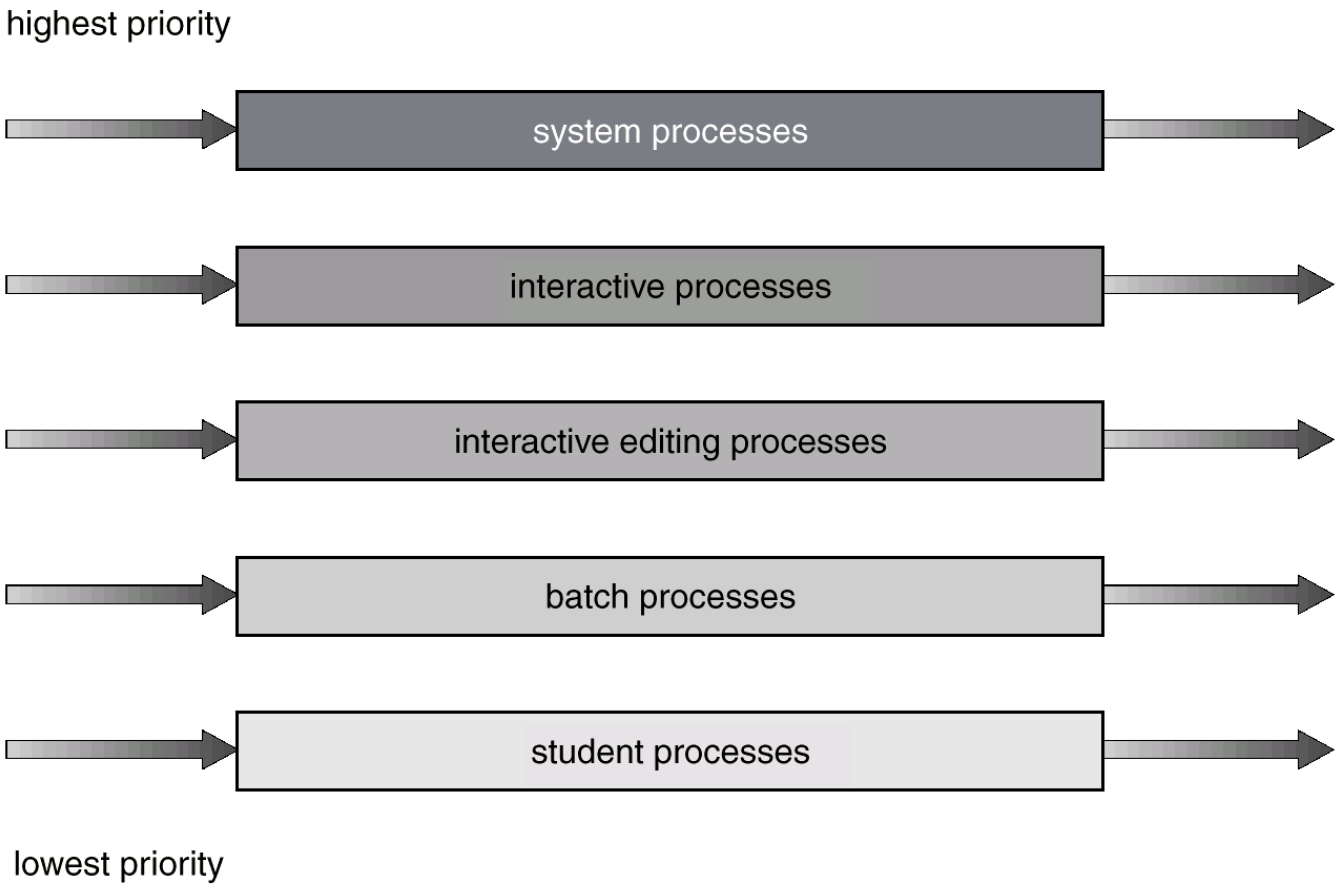


- Average waiting time = 8.2 msec

Multilevel Queue

- **Ready queue is partitioned into separate queues:**
  foreground (interactive)
  background (batch)
- **Each queue has its own scheduling algorithm,**
  foreground – RR
  background – FCFS
- **Scheduling must be done between the queues.**
  - Fixed priority scheduling; i.e., serve all from foreground then from background. Possibility of starvation.
  - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e.,
    80% to foreground in RR
  - 20% to background in FCFS

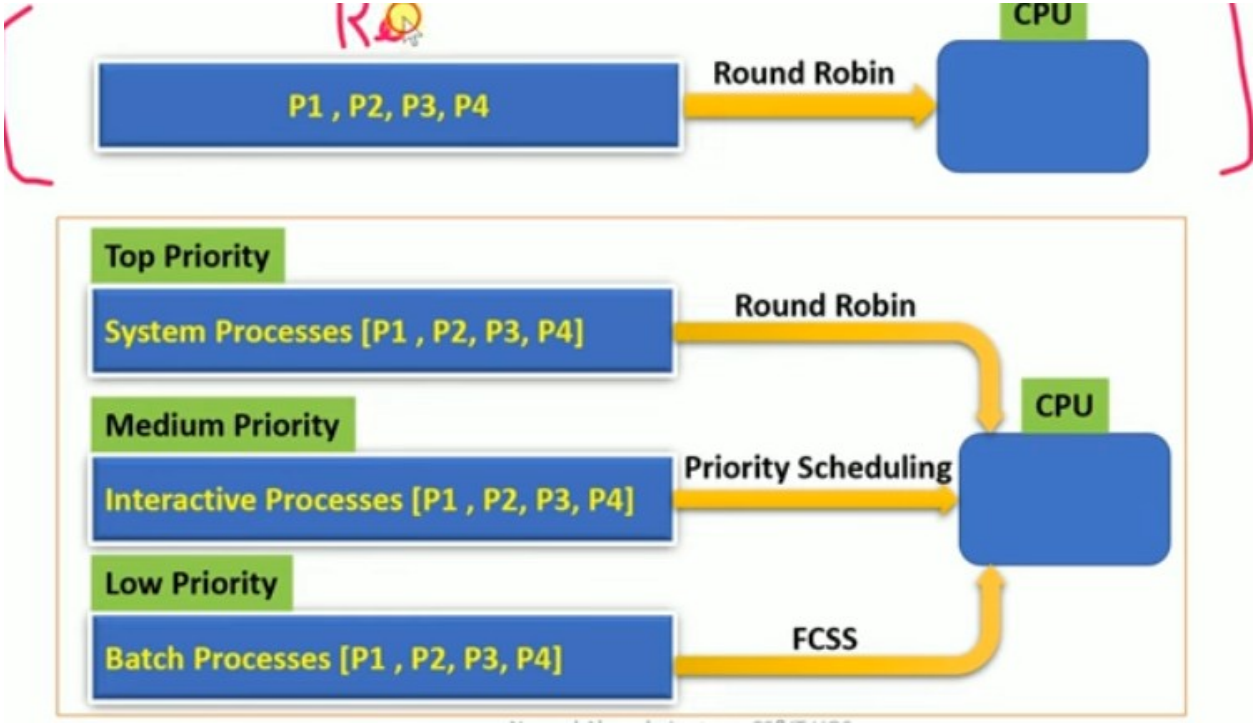
Multilevel Queue Scheduling



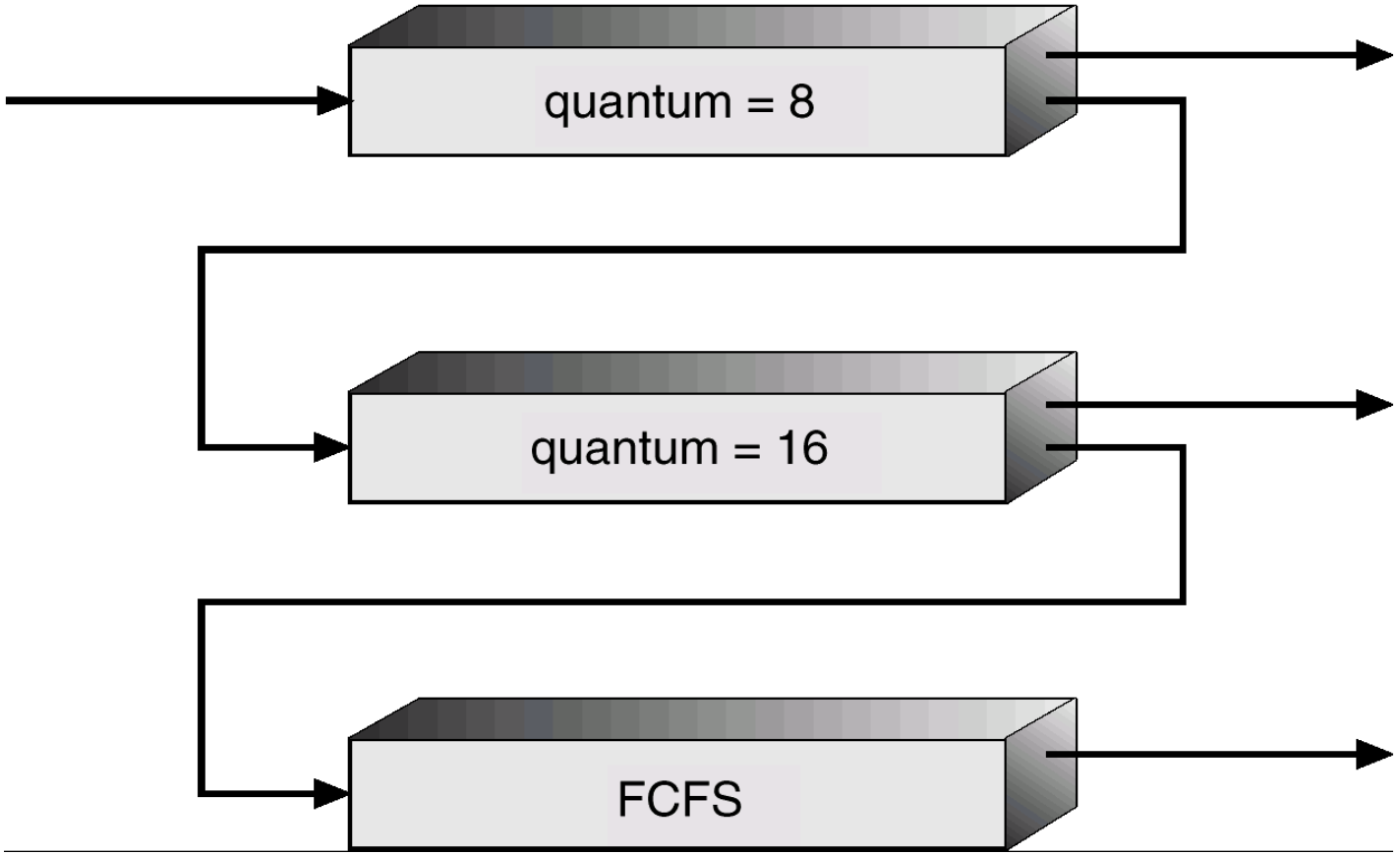
Multilevel Feedback Queue

- **A process can move between the various queues; aging can be implemented this way.**
- **Multilevel-feedback-queue scheduler defined by the following parameters:**
  - number of queues
  - scheduling algorithms for each queue
  - method used to determine when to upgrade a process
  - method used to determine when to demote a process
  - method used to determine which queue a process will enter when that process needs service

Example


CPU
P1 , P2, P3, P4
Round Robin
Top Priority
System Processes [P1 , P2, P3, P4]
Round Robin
CPU
Medium Priority
Interactive Processes [P1 , P2, P3, P4]
Priority Scheduling
Low Priority
Batch Processes [P1 , P2, P3, P4]
FCSS

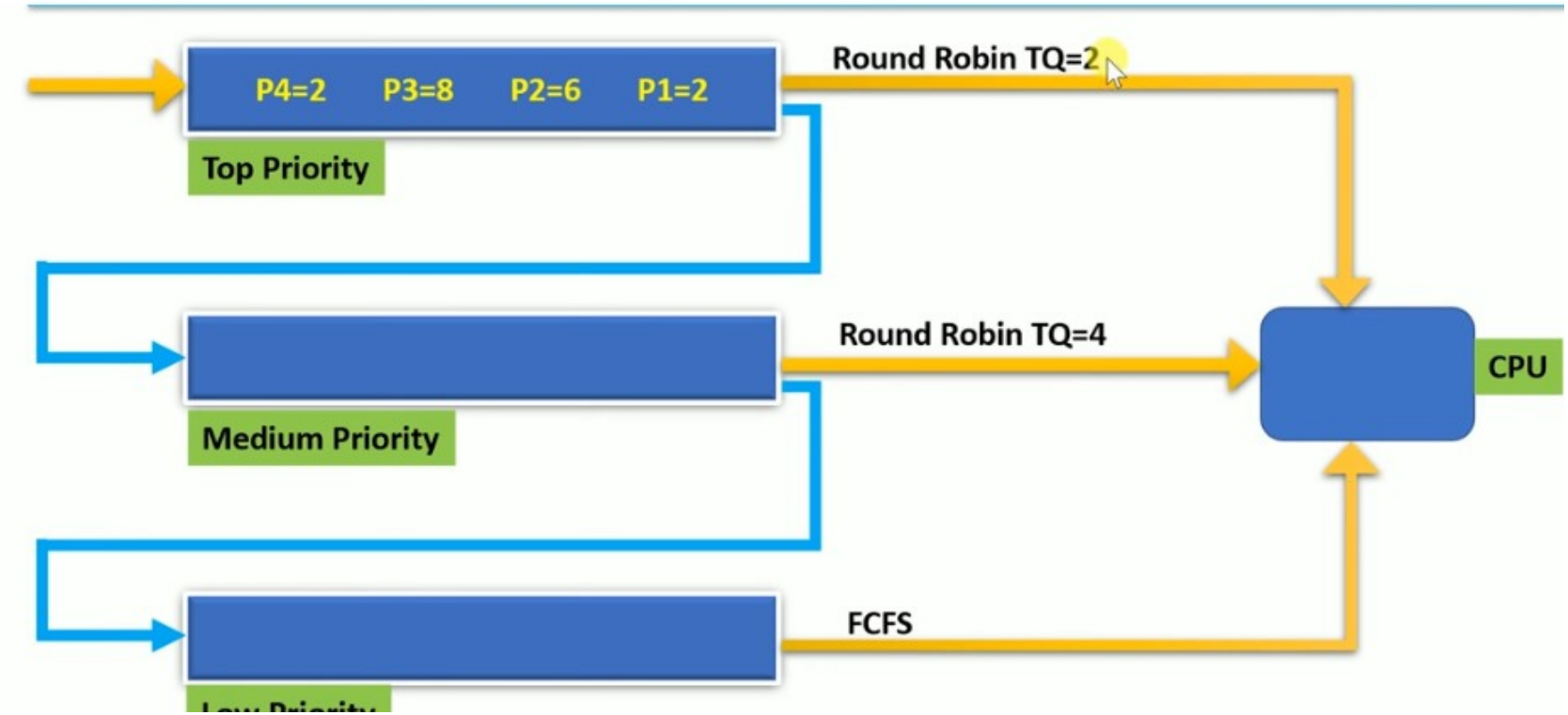
Multilevel Feedback Queues



Example of Multilevel Feedback Queue

- **Three queues:**
  - Q_0 – time quantum 8 milliseconds
  - Q_1 – time quantum 16 milliseconds
  - Q_2 – FCFS
- **Scheduling**
  - A new job enters queue Q_0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q_1.
  - At Q_1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2.

Example


P4=2 P3=8 P2=6 P1=2
Round Robin TQ=2
Top Priority
Round Robin TQ=4
Medium Priority
CPU
FCFS